

Atividade Acadêmica: Engenharia de Software – Arquitetura e Padrões
Professor: Samuel Armbrust Freitas (samuelaf@unisinos.br)

Trabalho T2

Definição do Trabalho

O objetivo deste trabalho é que os alunos projetem, implementem e documentem uma API RESTful utilizando boas práticas de Arquitetura de Software. O projeto deve ser realizado em *equipes de até 4 pessoas* e contemplar aspectos como modularização, padrões arquiteturais, testes automatizados e documentação adequada.

Descrição do Trabalho

As equipes deverão desenvolver uma API para um sistema de *Gestão de tarefas Colaborativas*, permitindo que usuários criem, editem, atribuam e concluam tarefas. A API deve seguir uma arquitetura bem definida (por exemplo, Arquitetura Hexagonal, Clean Architecture ou MVC), garantindo boas práticas de desacoplamento e modularização.

Além da implementação, é necessário entregar uma documentação técnica detalhando as decisões arquiteturais e um conjunto de testes automatizados para garantir a confiabilidade da API.

Requisitos Funcionais

A API deve expor os seguintes *endpoints*:

Usuários

- POST /users → Criar um novo usuário
- GET /users/{id} → Obter informações de um usuário específico
- PUT /users/{id} → Atualizar informações do usuário
- DELETE /users/{id} → Remover um usuário (soft delete recomendado)

Tarefas

- POST /tasks → Criar uma nova tarefa
- GET /tasks/{id} → Obter detalhes de uma tarefa
- GET /tasks?assignedTo={userId} → Listar todas as tarefas atribuídas a um usuário
- PUT /tasks/{id} → Atualizar informações da tarefa (título, descrição, status)
- DELETE /tasks/{id} → Remover uma tarefa

Autenticação

- POST /auth/login → Login de usuários, retornando um token JWT para autenticação nas demais requisições
- POST /auth/logout → Logout do usuário

Requisitos Complementares

Com o objetivo de aumentar o desafio técnico, a equipe pode escolher 2 dos requisitos complementares listados a seguir para a implementação.

1 - Integração com Calendários (Google Calendar, Outlook)

- Permitir que tarefas atribuídas com datas sejam sincronizadas com o calendário do usuário
- Exemplo: ao criar uma tarefa com data e hora, o sistema cria um evento no Google Calendar do usuário

2 - Webhooks para Integração com Slack/Discord

- Disparar notificações sempre que uma tarefa for criada, atribuída ou concluída



- Pode ser implementado de forma genérica: usuários configuram seus próprios *webhooks*

3 - Integração com Serviços de Armazenamento (Google Drive, Dropbox)

- Permitir anexar arquivos às tarefas através de links autenticados

4 - Visualizações de Dados e Métricas

- *Dashboard de Produtividade*
 - Exposição de métricas por *endpoint* como:
 - Número de tarefas por status (pendente, em andamento, concluída)
 - Tarefas por usuário
 - Tempo médio de conclusão de tarefas
 - Pode ser um *endpoint /metrics* ou uma rota adicional que consuma esses dados com uma lib como o Chart.js
- *Exportação de Relatórios (CSV/PDF)*
 - Permitir que os usuários exportem suas tarefas e atividades realizadas
 - Ideal para times que precisam prestar contas ou fazer retrospectivas

5 - Sistema de Permissões com Papéis

- Definir papéis como "Administrador", "Usuário", "Convidado", com permissões específicas por *endpoint*
- Exemplo: apenas administradores podem deletar tarefas de outros usuários

6 - Comentários em Tarefas (Sub-recursos aninhados)

- Implementar `/tasks/{id}/comments` com suporte a:
 - Criação, leitura e exclusão de comentários
 - Associar comentários a usuários e timestamps

7 - Sistema de Notificações Internas (e-mail ou push)

- Notificar usuários sobre tarefas atribuídas ou alteradas
- Pode usar serviços como SendGrid, Mailgun, ou notificações simuladas via log

8 - Filtro Avançado de Tarefas

- Endpoint com múltiplos parâmetros:
GET `/tasks?status=done&priority=high&dueBefore=2025-06-01`

Requisitos Não Funcionais

- Utilizar linguagem de programação à escolha da equipe (exemplo: Python com FastAPI, Java com Spring Boot ou Node.js com Express)
- Implementar padrões arquiteturais claros (por exemplo, Arquitetura Hexagonal, MVC ou *Clean Architecture*)
- Utilizar banco de dados relacional (MySQL, PostgreSQL) ou NoSQL (MongoDB), justificando a escolha
- Implementar testes automatizados com cobertura mínima de *60% do código da API*
- Implementar logs e tratamento de erros adequados
- Documentar a API utilizando Swagger/OpenAPI



Documentação

A equipe deve entregar uma documentação contendo:

- Visão Geral: Objetivo do sistema, contexto de uso e instruções de instalação
- Decisões Arquiteturais: Justificativa para a escolha da arquitetura e padrões aplicados
- Modelagem de Dados: Diagrama do banco de dados e descrição das tabelas/coleções
- Fluxo de Requisições: Descrição dos principais *endpoints* e exemplos de uso
- Configuração e *Deploy*: Guia de execução do projeto, incluindo dependências e configuração do ambiente
- Testes Automatizados: Estratégia utilizada e métricas de cobertura

Formato de Entrega

- Repositório Git público ou privado com acesso concedido ao professor
- Código-fonte estruturado e seguindo boas práticas de versionamento
- Documentação em formato Markdown (README.md) ou PDF
- Vídeo demo da solução de até 5 minutos
- Testes automatizados implementados e executáveis via terminal (exemplo: pytest para Python, JUnit para Java, Jest para Node.js)

Critérios de Avaliação

- Arquitetura e organização do código (25%)
- Implementação correta dos requisitos funcionais (25%)
- Qualidade dos testes automatizados (20%)
- Documentação técnica e justificativa de decisões (20%)
- Uso de boas práticas de desenvolvimento - versionamento, padrões, logs, segurança (10%)

Prazos

Entrega do repositório Git com código e documentação (Data definida pelo professor)

Apresentação do projeto, conforme sorteio, de até 15 minutos por equipe (Data definida pelo professor)